

LoRA plan: fine-tuning the vlm_05 crop judge

Status: exploration (written 2026-07-19, generalization section added 2026-08-03).

Execution steps live in `RUNB00K_LORA.md`. Dataset tooling is `tools/export_lora_dataset.py` plus `tools/lora/`. For how this fits into a publishable evaluation, see [docs/PUBLIC_DATASETS.md](#).

A note on vocabulary, since this document mixes two fields. **LoRA** (Low-Rank Adaptation) is a fine-tuning method that freezes the original model weights and trains a small set of extra matrices alongside them, so an 8-billion-parameter model can be adapted on a single consumer GPU. **QLoRA** is the same thing with the frozen base model held in 4-bit precision to save memory. The **adapter** is the small trained file that comes out; it is useless on its own and is either loaded next to the base model or merged into it.

Why this, why now

The measured bottleneck is the judge, not the localizer. In `vlm_05`, the classical computer-vision stage that finds changed regions currently recovers all 45 of the 45 ground-truth objects in the benchmark. The problem is the second stage: when the vision-language model is asked whether each proposed region really contains an anomaly, only 0.52 to 0.66 of the regions it accepts are genuine, depending on which model and prompt are used. The errors are not random, they repeat as recognisable templates: "Blue seat cushion slightly shifted", "Black cable snake-like", the equipment bag, the onboard information display. Prompt engineering has stopped paying off. The lenient-versus- conservative prompt comparison showed the effect is small next to these false-positive families, and it varies from one model to another.

The literature says fine-tuning is what fixes this on small models. Borodin et al. (J. Imaging, 2025; referenced here as `CCTV_VLM_paper.pdf`) found that a rank-8 LoRA makes compact vision-language models competitive with much larger ones on surveillance anomaly detection, and that it also flattens their sensitivity to prompt wording. Those are exactly our two problems.

We now have a way to produce training labels cheaply. ARSI Studio's Review mode already asks a human to mark each proposed region as a true or false positive, and lets the reviewer draw boxes the pipeline missed. The export tool turns those verdicts into training samples that are rendered exactly the way inference renders them, using `vlm_05.render_crop_pair`, the current prompt, and `YES <label>` or `NO` as the expected answer.

What gets trained

Only the `vlm_05` crop judge, on top of Qwen3-VL-8B-Instruct. Not the localizer, which is classical computer vision and already finds everything. Not the whole-frame pipelines.

Qwen3-VL was chosen over GLM as the base for two practical reasons: its fine-tuning toolchain (LLaMA-Factory and Unsloth) is mature, and the authors publish official deployment artifacts. There is also an upside if it works. GLM is currently used as a second model for the frame-level alarm decision, so a tuned Qwen3-VL that beats GLM on region precision would let us run one model instead of two.

Dataset design

DECISION	CHOICE	REASON
What one sample looks like	the reference crop and the inspection crop side by side, with the current prompt	byte-for-byte identical to what inference sends, so training and deployment cannot drift apart. <code>stats.json</code> records a hash of the prompt used
Answer for a real anomaly	<code>YES <the label the human wrote></code>	keeps the model able to name the object, which the inventory feature needs
Answer for a false positive	a bare <code>NO</code>	this is what the output parser expects, and it avoids teaching the model to invent justifications
Where samples come from	human review verdicts and reviewer-drawn missed boxes	training on the model's own verdicts would be distillation, which copies its mistakes instead of correcting them
Train/validation split	90/10, split by frame	all crops taken from one frame stay on the same side of the split
Benchmark ground truth	excluded by default	the 29-case benchmark is our only honest evaluation. A <code>--include-benchmark</code> flag exists, but using it is irreversible: once those cases are in training, the benchmark no longer measures anything
Balance guard	warns if the ratio of <code>YES</code> to <code>NO</code> falls outside 1:5 to 5:1	a burst of reviews that only marks false positives would teach the model to always answer <code>NO</code>

On volume: aim for somewhere between 300 and 500 crops. The Borodin paper saw gains at that scale on a task narrower than ours, so treat it as a floor rather than a target, and keep labelling as a side effect of normal Studio use.

Training recipe (sized for the 12 GB RTX 3080 Ti)

QLoRA at 4-bit, rank 8, alpha 16, applied to all the linear layers of the language model. The **vision tower and the projector stay frozen**, meaning only the text side of the model learns. Batch size 1 with gradients accumulated over 8 steps, learning rate 1e-4 on a cosine schedule, at most 3 epochs, checking validation loss after each one.

Freezing the vision side is not just the standard recommendation for domain adaptation. It also matters at deployment: because only the text backbone changes, the multimodal projector file published by the model authors (the `mmproj`, which is what turns image features into something the language model can read) stays valid and can be reused as is. An 8-billion-parameter QLoRA run at these settings is documented to fit in roughly 10 to 12 GB. If LLaMA-Factory runs out of GPU memory, Unsloth is the fallback.

Deployment path

The chain is: merge the adapter into the base weights, save as 16-bit floats in Hugging Face format, convert with `convert_hf_to_gguf.py` into GGUF (the file format llama.cpp and Ollama both read), quantize to Q4_K_M, then pair the result with the **official** Qwen3-VL projector file rather than a regenerated one.

- 01 **First try importing into Ollama** with a Modelfile that has two `FROM` lines, one for the model and one for the projector. The risk is real: Ollama's import path for vision models has a history of silently dropping the projector, which produces a model that loads fine and answers text questions fine while being completely blind (see Ollama issues #9967 for Gemma 3 and #14730 for qwen35moe). So test it with a question that cannot be answered without looking at the image, and treat a plausible text-only answer as a failure, not a pass.
- 02 **If that fails, run `llama-server --mmproj` instead.** This path is known to work. It needs about 30 lines of adapter code to expose an OpenAI-compatible endpoint, injected through `OllamaClient(impl=...)`. That injection point already exists in the codebase and is what the tests use.

Evaluation protocol

Run the untouched 29-case benchmark with a cleared cache, using the tuned model as the `vlm_05` judge. This is set up identically to the base-model sweeps, so the resulting report can be compared row by row against `report_lenient_qwen3vl.md` and the GLM report.

Go/no-go criterion: region precision must improve by at least 0.10 over the base Qwen3-VL, object recall must not drop by more than 0.02, and the output format must still be intact on a 20-crop spot check. Secondary checks: whether the frame-level decision stays specific on the 5 cross-session negatives, and, once that run exists, the false-alarm rate on the two videos of an empty tram recorded on the RTX setup.

Generalization: what the frame-level split does not buy us

The 90/10 split is by frame. That stops crops from a single frame appearing on both sides of the split, and it does nothing else. In particular it gives no protection against overfitting to a session or to a camera. The failure mode to worry about looks like this:

training set	F1 = 0.97
same session, held out	F1 = 0.91
a different vehicle	F1 = 0.52

which is what you get when the adapter has quietly learned this tram's seat fabric, this camera's vignetting and this lighting, rather than the concept "an object is present that was not there before".

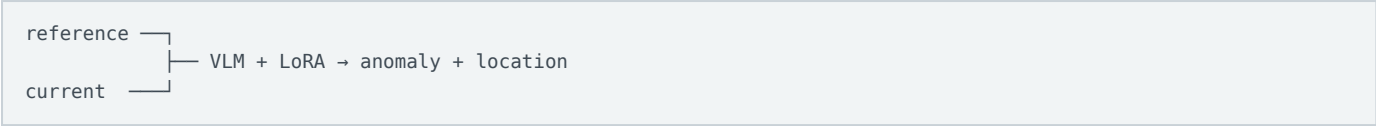
Our current data makes this a live concern rather than a theoretical one. Everything under `data/raw` comes from a single vehicle, tram 1762, in four recording sessions `v1` through `v4`, plus one `tram_variant_reference.png`. There is no second vehicle and no second camera position. So generalization across vehicles cannot be measured today at all. That is a limitation to write down plainly, in the paper and in any report to VSB, not something to work around.

What we can still do with the data we have:

- **Keep one full session out of training**, `v4` unless review coverage makes another choice better, and report its numbers separately from the 90/10 validation loss. Breaking region precision down per session is the cheapest early warning available.
- **Leave the 5 cross-session negatives and the two empty-tram videos alone.** They are already the closest thing we have to a test under unseen conditions, and they lose that value the moment anything from them enters training.
- **Treat a large gap between the held-out session and the validation numbers as a no-go in its own right**, separately from the +0.10 precision target. A tuned judge that only works on `v1` through `v3` is worse than the base model, because it looks like it works.
- **Push for a second tram, or at least a second camera mounting.** This is the single most valuable data we could acquire, both for the go/no-go decision and for the paper. It should be raised with VSB early, because arranging access to a vehicle takes time.

An open research question (for the paper, not for the go/no-go)

Once the crop judge is working, there is a third architecture worth trying: give the fine-tuned model the reference image and the current image directly and let it decide for itself, with no explicit differencing stage in front of it.



Stated as a question: **can a compact vision-language model, adapted to this domain, replace explicit image differencing for detecting persistent anomalies?** That is something a reviewer will find interesting.

It also fills in the ablation table in [docs/PUBLIC_DATASETS.md](#) :

	FULL FRAME	DIFFERENCING THEN CROP
off-the-shelf model	A	C
fine-tuned model	B	D

The plan as written produces D, and the sweeps we already have give us C. Adding A and B is cheap and turns the table into an actual measurement: how much of the performance comes from the change-detection front end, and how much from adapting the model to the domain. This should not reorder the work. The crop judge is the deliverable; this is the experiment that explains why it works.

Risks and open questions

- **Template or library drift.** Confirm that the `qwen3_vl` template identifier still matches the installed version of LLaMA-Factory. There is a check for this at the top of the YAML config.
- **The dataset is small.** Overfitting will show up as validation loss turning upward and as the output format breaking down. The runbook checks for both.
- **The adapter is tied to the current prompt.** It is trained against one exact prompt string, whose hash is recorded in `stats.json` . Changing the prompt later means re-exporting the dataset and retraining.
A corollary, because this suggestion keeps coming up: **do not switch the training target to structured JSON** such as `{"anomaly": true, "type": ..., "location": ...}` . The `YES <label>` and bare `NO` format is what the parser already consumes and is byte-for-byte identical to inference. A richer output format buys nothing at this stage and costs a full re-export, plus it invites the model to invent justifications we have no way to verify.
- **Merging a 4-bit-trained adapter onto 16-bit weights** is standard practice, but it is known to cost a small and unpredictable amount of quality. The benchmark comparison is what settles whether it mattered.
- **If the Ollama import fails and llama-server turns out to be a nuisance to operate**, it is perfectly acceptable to run only the `vlm_05` judge through llama-server and leave everything else on Ollama. They are separate processes and do not need to agree.

